



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №9

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 Сохранение состояния приложения	3
1.2 Хранение данных в Android	4
1.3 Внутреннее хранилище.....	6
1.3.1 Создание и запись файла	6
1.3.2 Чтение файла	7
1.3.3 Удаление файла	8
1.4 Внешнее хранилище	8
1.4.1 Создание и запись файла	10
1.4.2 Чтение файла	11
1.4.3 Удаление файла	11
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	12
2.1 Работа с внутренним хранилищем	12
2.1.1 Создание файлов	12
2.1.2 Чтение файлов	13
2.1.3 Удаление файлов	14
2.1.4 Добавление строк в конец файла.....	16
2.2 Сохранение состояния приложения при повороте экрана.....	17
2.3 Работа с файлами в различных приложениях	19
2.3.1 Приложения для записи файлов	20
2.3.2 Приложение для считывания данных из файлов	24
ЗАКЛЮЧЕНИЕ	27

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 Сохранение состояния приложения

Сохранение и восстановление состояния приложения являются важными аспектами разработки Android-приложений, особенно при обработке изменений конфигурации, таких как поворот экрана, изменение языка и других сценариев, которые приводят к пересозданию активности.

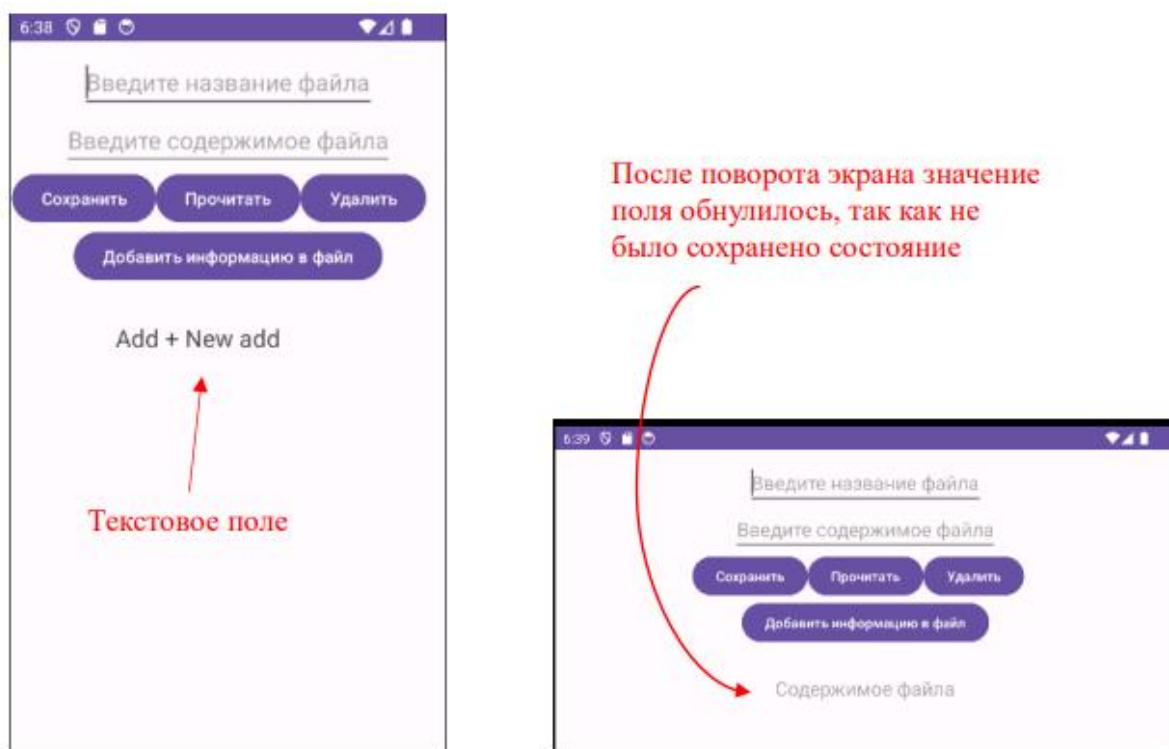


Рисунок 1.1.1 – Отображение приложения при повороте экрана

Android предоставляет несколько механизмов для управления состоянием приложения, включая использование методов `onSaveInstanceState()` и `onRestoreInstanceState()`. Эти методы позволяют сохранять и восстанавливать данные о состоянии пользовательского интерфейса, обеспечивая бесперебойное взаимодействие пользователя с приложением.

Метод `onSaveInstanceState()` вызывается системой перед тем, как активность будет уничтожена, чтобы дать возможность сохранить состояние пользовательского интерфейса в объект `Bundle` (рисунок 1.1.2). В

этот объект можно добавлять различные типы данных, такие как строки, числа, сериализуемые объекты и другие.

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    // Сохраняем значение строковой переменной
    outState.putString("KEY_STATE", "some state");
}
```

Рисунок 1.1.2 – Переопределение метода onSaveInstanceState

В методе onSaveInstanceState() сохраняем состояние. Для этого вызываем у параметра Bundle метод putString(key, value), первый параметр которого - ключ, а второй - значение сохраняемых данных.

Метод onRestoreInstanceState() вызывается после метода onStart(), когда активность воссоздается после пересоздания. Этот метод получает объект Bundle, содержащий данные о состоянии, которые были сохранены в onSaveInstanceState() (рисунок 1.1.3).

```
@Override
protected void onRestoreInstanceState(Bundle savedInstanceState) {
    super.onRestoreInstanceState(savedInstanceState);
    // Восстанавливаем сохраненное состояние
    String state = savedInstanceState.getString(key: "KEY_STATE");
    // Используем сохраненное значение для восстановления состояния UI или других компонентов
}
```

Рисунок 1.1.3 – Переопределение метода onRestoreInstanceState

Таким образом, например, при смене ориентации экрана все записанные данные не пропадут.

1.2 Хранение данных в Android

Более удобным способом сохранять все состояния приложения является их запись в отдельный файл, который не будет затронут при смене метода жизненного цикла активности.

Работа с файловой системой в Android является ключевым элементом в процессе разработки мобильных приложений. Это необходимость

обусловлена разнообразными задачами, с которыми сталкиваются разработчики: от простого сохранения настроек пользователя и состояний приложения до сложных операций с медиафайлами, документами и другими данными, требующими постоянного хранения между сессиями. Кроме того, эффективное использование файловой системы позволяет реализовывать функции загрузки ресурсов из сети и их последующего кэширования, что существенно улучшает производительность приложения и удобство его использования.

В Android предусмотрены два основных типа хранилища для работы с файлами: внутреннее (рисунок 1.2.1) и внешнее. Внутреннее хранилище: это хранилище, доступное пользователю только через установленные приложения (или при наличии root-доступа на устройстве). Пример: `data/data/app_packageName`.

Внешнее хранилище бывает двух типов:

- основное внешнее хранилище: встроенное общее хранилище, к которому «пользователь может получить доступ, подключив USB-кабель и установив его как накопитель на хост-компьютере». Пример: когда мы говорим о Nexus 5 с 32 ГБ памяти;
- вторичное внешнее хранилище: съемное хранилище. Пример: SD-карта.

Внутреннее хранилище гарантирует безопасность данных, делая их доступными только для вашего приложения, и обеспечивает их удаление при деинсталляции приложения. Внешнее хранилище, в свою очередь, предлагает больший объем памяти и возможность обмена файлами между приложениями и даже передачу данных между устройствами.

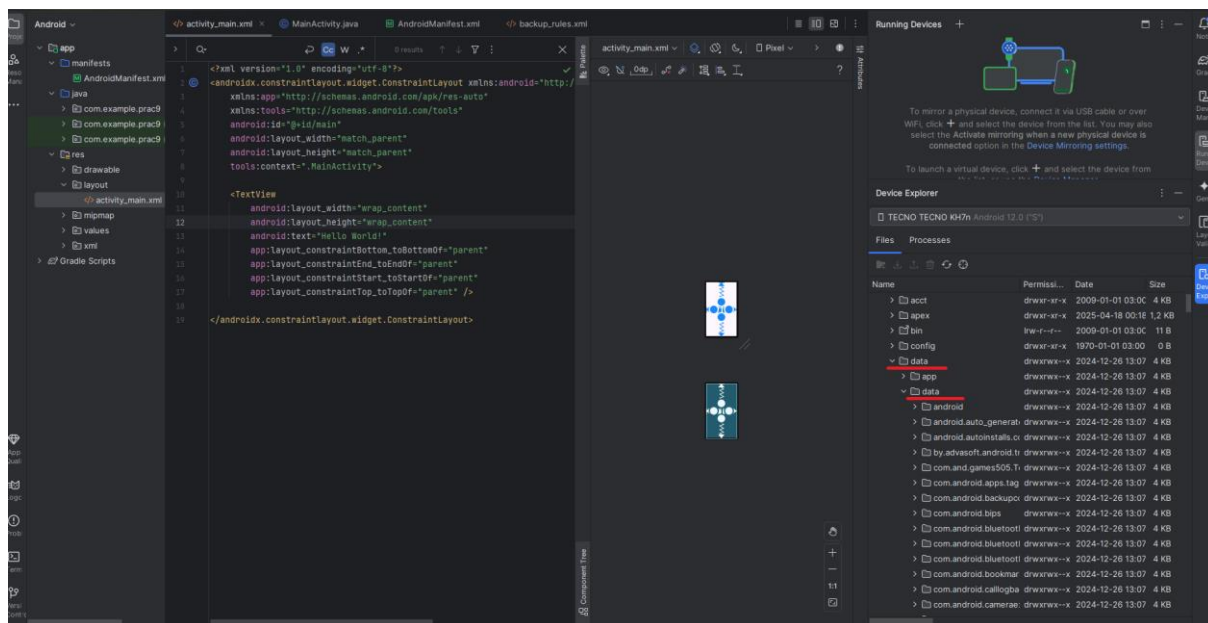


Рисунок 1.2.1 – Расположение внутренней памяти в Android

1.3 Внутреннее хранилище

Внутреннее хранилище предназначено для индивидуальных данных приложения, к которым не предполагается доступ из других приложений. Данные, сохраненные во внутреннем хранилище, удаляются при удалении приложения. Для работы с внутренним хранилищем используются различные методы.

1.3.1 Создание и запись файла

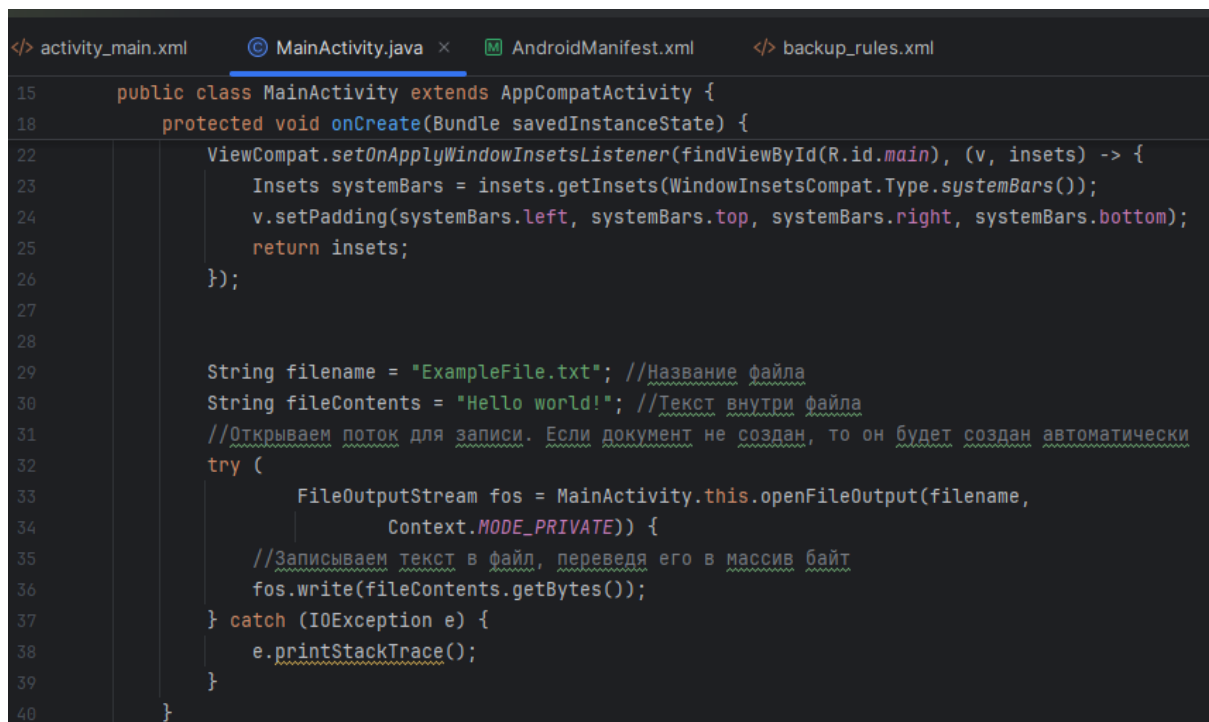
Для создания файла требуется открыть поток для его записи/чтения.

Система позволяет создавать файлы с двумя разными режимами:

- **MODE_PRIVATE**: файлы могут быть доступны только владельцу приложения (режим по умолчанию);
- **MODE_APPEND**: данные могут быть добавлены в конец файла.

Если файл уже существует, то он будет перезаписан. Если же нам надо дописать файл, тогда надо использовать режим **MODE_APPEND**. Для автоматического закрытия файла и освобождения ресурса объект

FileOutputStream создается с помощью конструкции try...catch (рисунок 1.3.1.1).



```
15 public class MainActivity extends AppCompatActivity {
16     protected void onCreate(Bundle savedInstanceState) {
17         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
18             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
19             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
20             return insets;
21         });
22
23         String filename = "ExampleFile.txt"; //Название файла
24         String fileContents = "Hello world!"; //Текст внутри файла
25         //Открываем поток для записи. Если документ не создан, то он будет создан автоматически
26         try {
27             FileOutputStream fos = MainActivity.this.openFileOutput(filename,
28                 Context.MODE_PRIVATE);
29             //Записываем текст в файл, переводя его в массив байт
30             fos.write(fileContents.getBytes());
31         } catch (IOException e) {
32             e.printStackTrace();
33         }
34     }
35 }
```

Рисунок 1.3.1.1 – Создание текстового файла

В итоге весь текст будет сохранен в файле
/data/data/название_пакета/files/ExampleFile.txt

И если дважды нажать на файл, то можно увидеть его содержимое

1.3.2 Чтение файла

Класс FileInputStream создаёт объект класса InputStream, который можно использовать для чтения байтов из файла.

InputStreamReader – это переход от потоков байтов к потокам символов: он считывает байты и декодирует их в символы, используя указанный Charset. Используемая им кодировка может быть указана по имени или может быть задана явно, или может быть принята кодировка платформы default charset (рисунок 1.3.2.1).

```

1 usage
public void read_file(View v){
    String file_name = file_name_text.getText().toString();
    try (FileInputStream fis = MainActivity.this.openFileInput(file_name)){
        InputStreamReader inputStreamReader = new InputStreamReader(fis,
            StandardCharsets.UTF_8);
        StringBuilder stringBuilder = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(inputStreamReader)){
            String line = reader.readLine();
            while (line != null){
                stringBuilder.append(line).append('\n');
                line = reader.readLine();
            }
        } catch (IOException e){
            e.printStackTrace();
        }
        String contents = stringBuilder.toString();
        file_read_data.setText(contents);
    } catch (IOException e){
        e.printStackTrace();
    }
}
}

```

Рисунок 1.3.2.1 – Реализация чтения текста из файла

1.3.3 Удаление файла

Для того чтобы удалить файл необходимо обратиться к его директории, а после провести удаление самого файла (рисунок 1.3.3.1).

```

1 usage
public void delete_file(){
    String file_name = file_name_text.getText().toString();
    File dir = getFilesDir();
    File file = new File(dir, file_name);
    boolean deleted = file.delete();
    Toast.makeText(context: this, text: deleted + "", Toast.LENGTH_LONG).show();
}

```

Рисунок 1.3.3.1 – Реализация удаления файла

1.4 Внешнее хранилище

В прошлой теме были рассмотрены сохранение и чтение файлов из каталога приложения. По умолчанию такие файлы доступны только самому приложению.

Внешнее хранилище используется для сохранения файлов, которые могут быть общими для нескольких приложений или требуют сохранения даже после удаления вашего приложения. Прежде чем создавать файлы, убедитесь, что у вашего приложения есть необходимые разрешения для работы с внешним хранилищем. Начиная с Android 6.0 (API уровня 23), требуется запросить эти разрешения во время выполнения (для этого добавьте в файл Манифеста разрешения READ_EXTERNAL_STORAGE и WRITE_EXTERNAL_STORAGE) (рисунок 1.4.1).

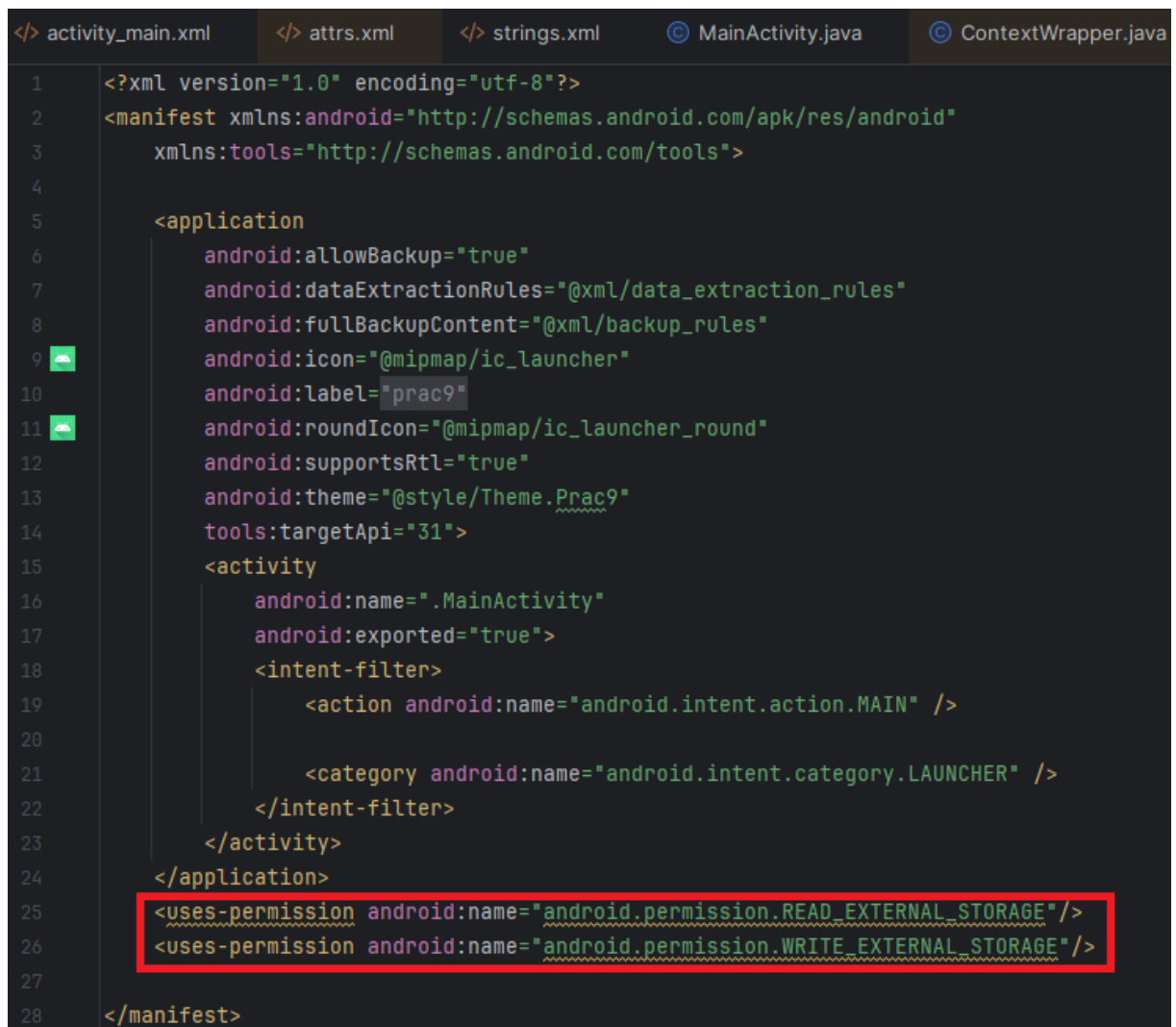


Рисунок 1.4.1 – Добавление разрешений на чтение и запись файлов внешнего хранилища

Однако способ, представленный на рисунке 1.4.1 не предоставляет доступ к файлам внешнего хранилища, начиная с версии API 29, что довольно печально.

Для Android 10 и выше рекомендуется использовать механизм Scoped Storage, который не требует явных разрешений для доступа к определенным типам файлов, таким как фотографии и видео, через MediaStore API.

1.4.1 Создание и запись файла

Для создания файла в первую очередь необходимо удостовериться в существовании заданного репозитория, в котором хранится выбранный файл, после чего происходит создание файла с записью или без (рисунок 1.4.1.1).

```
public void save_file(View v){
    String file_name = file_name_text.getText().toString();
    File dir = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);
    if (!dir.exists()){
        dir.mkdirs();
    }
    File file = new File(dir, file_name);
    try{
        if (!file.exists()){
            boolean created = file.createNewFile();
            if (created){
                FileWriter writer = new FileWriter(file);
                writer.append(file_data_text.getText().toString());
                writer.flush();
                writer.close();
            }
        }
        else{
            boolean readable = file.canRead();
            boolean writeable = file.canWrite();
        }
    }catch (IOException e){
        e.printStackTrace();
    }
}

1 usage
public void add_data(View v) throws IOException {
    String file_name = file_name_text.getText().toString();
    File dir = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);

    if (!dir.exists()){
        dir.mkdirs();
    }

    File file = new File(dir, file_name);
    FileOutputStream fos = new FileOutputStream(file);
    fos.write(file_data_text.getText().toString().getBytes());
    fos.close();
}
```

Рисунок 1.4.1.1 – Реализация создания файла и записи в него текста во внешнем хранилище

1.4.2 Чтение файла

Для чтения файла необходимо убедиться в его существовании, после чего провести попытку получения доступа к чтению файла и в случае удовлетворительного результата воспроизвести чтение файла (рисунок 1.4.2.1).

```
public void read_file(View v){
    String file_name = file_name_text.getText().toString();
    File dir = getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);
    File file = new File(dir, file_name);

    if (file.exists()) {
        StringBuilder text = new StringBuilder();

        try {
            boolean readable = file.canRead();
            BufferedReader br = new BufferedReader(new FileReader(file));
            String line;
            while ((line = br.readLine()) != null) {
                text.append(line).append('\n');
            }
            file_read_data.setText(text);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Рисунок 1.4.2.1 – Реализация чтения файла из внешнего хранилища

1.4.3 Удаление файла

Для удаления файла необходимо обратиться к его родительской директории, а после к нему самому для его непосредственного удаления (рисунок 1.4.3.1).

```
1 usage
public void delete_file(){
    String file_name = file_name_text.getText().toString();
    File dir = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);
    File file = new File(dir, file_name);
    boolean deleted = file.delete();
    Toast.makeText(context, this, text: deleted + "", Toast.LENGTH_LONG).show();
}
```

Рисунок 1.4.3.1 – Реализация удаления файла из внешнего хранилища

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

Данная практическая работа была выполнена с API 28 и ниже. Связано это с тем, что `uses-permission` для получения разрешения к файлам внешнего и внутреннего хранилища необходимо работать на более старых версиях из-за их специфики работы с памятью.

2.1 Работа с внутренним хранилищем

2.1.1 Создание файлов

Для реализации всех последующих манипуляций с файлами необходимо описать фундамент их логики: соединение слушателей событий с элементами разметки, а также объявление необходимых полей (рисунок 2.1.1.1).

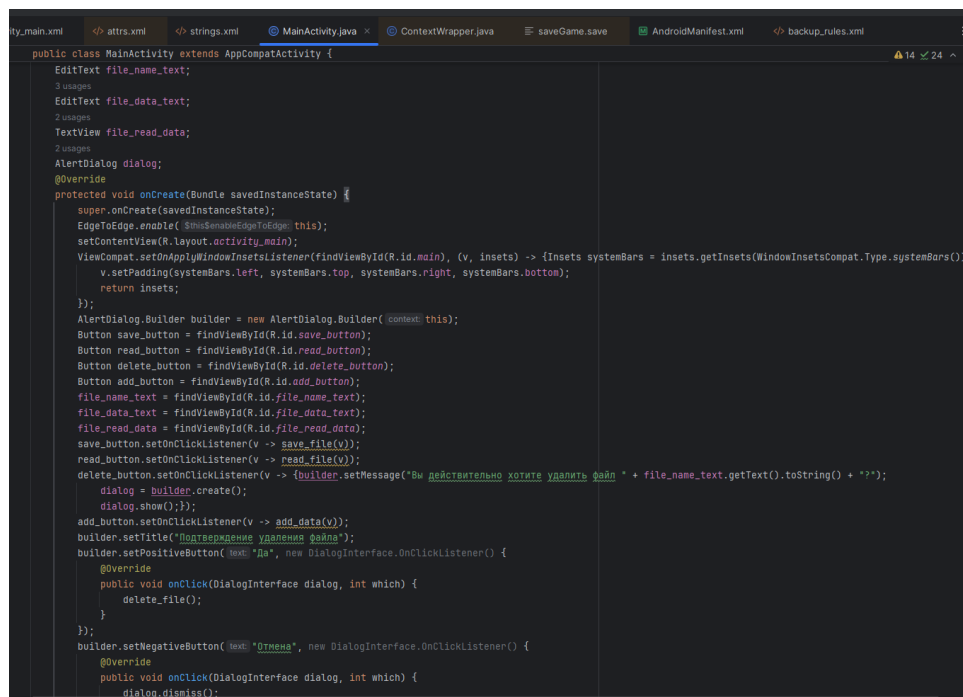


Рисунок 2.1.1.1 – Описание логики создания activity

При создании файла необходимо получить его имя, а после доступ к записи, если такой файл уже существует, и переписать его (рисунок 2.1.1.2).

Также была произведено испытание на проверку безошибочности кода (рисунок 2.1.1.3).

```

1 usage
public void save_file(View v){
    String file_name = file_name_text.getText().toString();
    try (FileOutputStream fos = this.openFileOutput(file_name,
        Context.MODE_PRIVATE)){
        fos.write(file_data_text.getText().toString().getBytes());
    } catch (IOException e){
        e.printStackTrace();
    }
}
}

```

Рисунок 2.1.1.2 – Описание логики создания файла во внутреннем хранилище

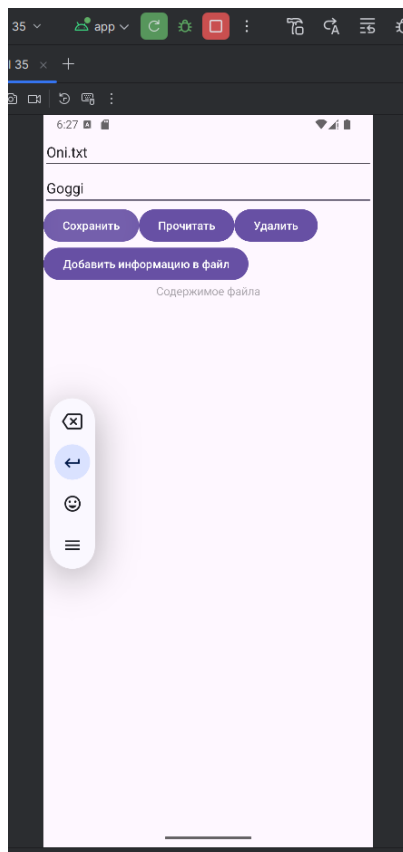


Рисунок 2.1.1.3 – Итоговый вид приложения

2.1.2 Чтение файлов

Для чтения строк из файлов воспользуемся объектом класса `StringBuilder`. Процесс заключается в следующем: поступает имя файла, происходит переход внутреннюю директорию приложения и в случае существования выбранного файла происходит его построчное считывание (рисунок 2.1.2.1). При этом необходимо вывести содержимое файла на экран (рисунок 2.1.2.2).

```

1 usage
public void read_file(View v){
    String file_name = file_name_text.getText().toString();
    try (FileInputStream fis = MainActivity.this.openFileInput(file_name)){
        InputStreamReader inputStreamReader = new InputStreamReader(fis,
            StandardCharsets.UTF_8);
        StringBuilder stringBuilder = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(inputStreamReader)){
            String line = reader.readLine();
            while (line != null){
                stringBuilder.append(line).append('\n');
                line = reader.readLine();
            }
        } catch (IOException e){
            e.printStackTrace();
        }
        String contents = stringBuilder.toString();
        file_read_data.setText(contents);
    } catch (IOException e){
        e.printStackTrace();
    }
}
}

```

Рисунок 2.1.2.1 – Реализация считывания строк из файлов во внутреннем хранилище

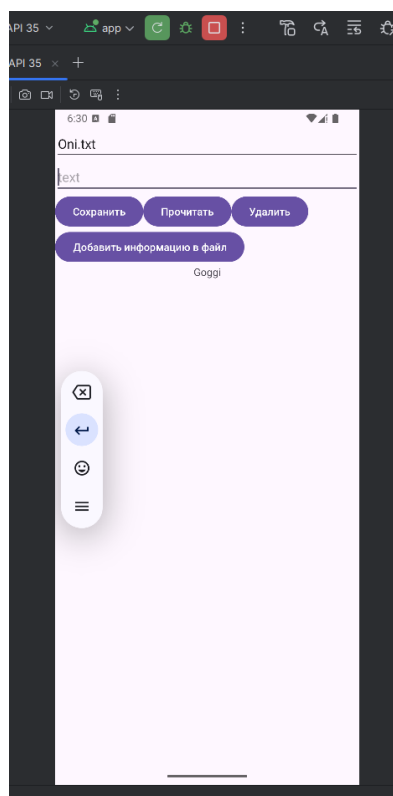


Рисунок 2.1.2.2 – Итоговый вид приложения

2.1.3 Удаление файлов

В случае удаления необходимо получить файл как и в предыдущие разы и произвести непосредственно удаление с выводом результата на экран (рисунок 2.1.3.1 – 2.1.3.2).

```

1 usage
public void delete_file(){
    String file_name = file_name_text.getText().toString();
    File dir = getFilesDir();
    File file = new File(dir, file_name);
    boolean deleted = file.delete();
    Toast.makeText(context: this, text: deleted + "", Toast.LENGTH_LONG).show();
}

```

Рисунок 2.1.3.1 – Реализация удаления файла

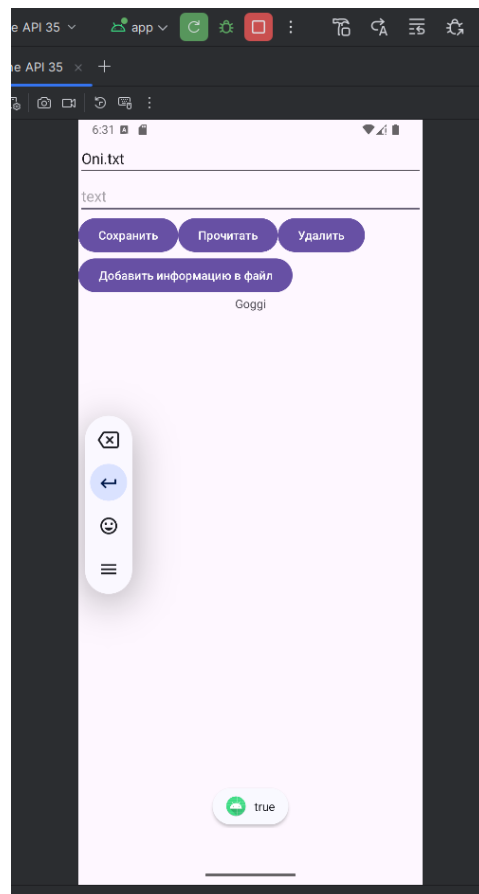


Рисунок 2.1.3.2 – Итоговый вид приложения

Во время этого процесса происходит получение разрешения пользователя на удаление файла в AlertDialog (рисунок 2.1.3.3).

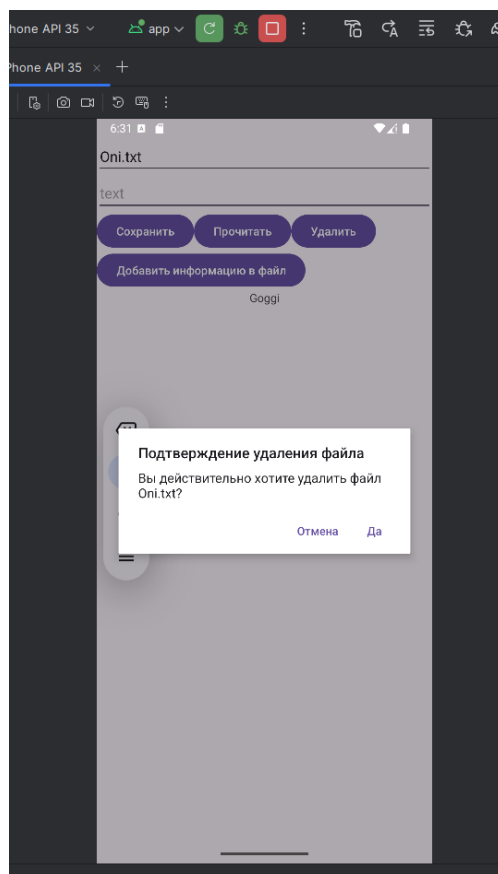


Рисунок 2.1.3.3 – Итоговый вид приложения

В случае отказа пользователя от удаления файла процесс удаления прерывается.

2.1.4 Добавление строк в конец файла

Процесс получения файла аналогичен предыдущим. В данном случае происходит дописывание строк в конец существующего файла или же создания нового с полученным текстовым значением (рисунок 2.1.4.1).

```
1 usage
public void add_data(View v){
    String file_name = file_name_text.getText().toString();
    try (FileOutputStream fos = this.openFileOutput(file_name,
        Context.MODE_APPEND)){
        fos.write(('n' + file_data_text.getText().toString()).getBytes());
    } catch (IOException e){
        e.printStackTrace();
    }
}
```

Рисунок 2.1.4.1 – Реализация добавления строкового значения в файл во внутренней памяти приложения

Далее следует проверка изменённого файла путём его прочтения и вывода на экран его содержимого (рисунок 2.1.4.2).

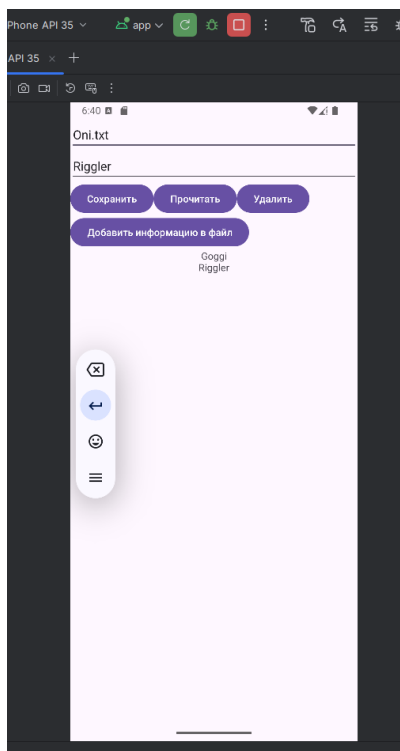


Рисунок 2.1.4.2 – Итоговый вид приложения

2.2 Сохранение состояния приложения при повороте экрана

Для сохранения состояния воспользуемся методом, вызываемым перед уничтожением activity, `onSaveInstanceState`, а также методом, вызываемым при пересоздании activity, `onRestoreInstanceState` (рисунок 2.2.1).

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    // Сохраняем значение строковой переменной
    outState.putString("KEY_STATE", "some state");
}

@Override
protected void onRestoreInstanceState(Bundle savedInstanceState) {
    super.onRestoreInstanceState(savedInstanceState);
    // Восстанавливаем сохраненное состояние
    String state = savedInstanceState.getString(key: "KEY_STATE");
    // Используем сохраненное значение для восстановления состояния UI или других компонентов
}
```

Рисунок 2.2.1 – Реализация сохранения данных при смене ориентации экрана

Далее была проведена проверка итогового функционала приложения (рисунок 2.2.2 – 2.2.3).

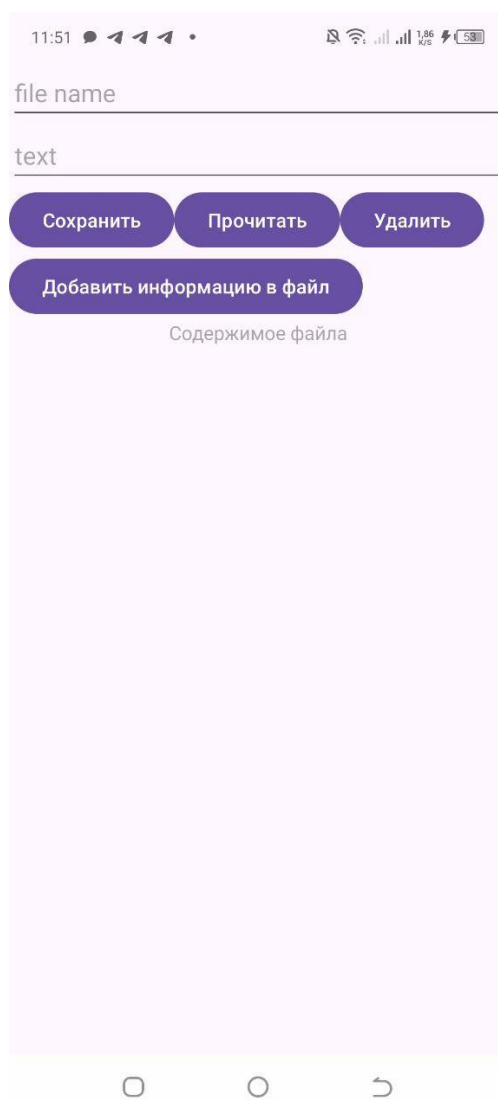


Рисунок 2.2.2 – Вертикальный вид приложения

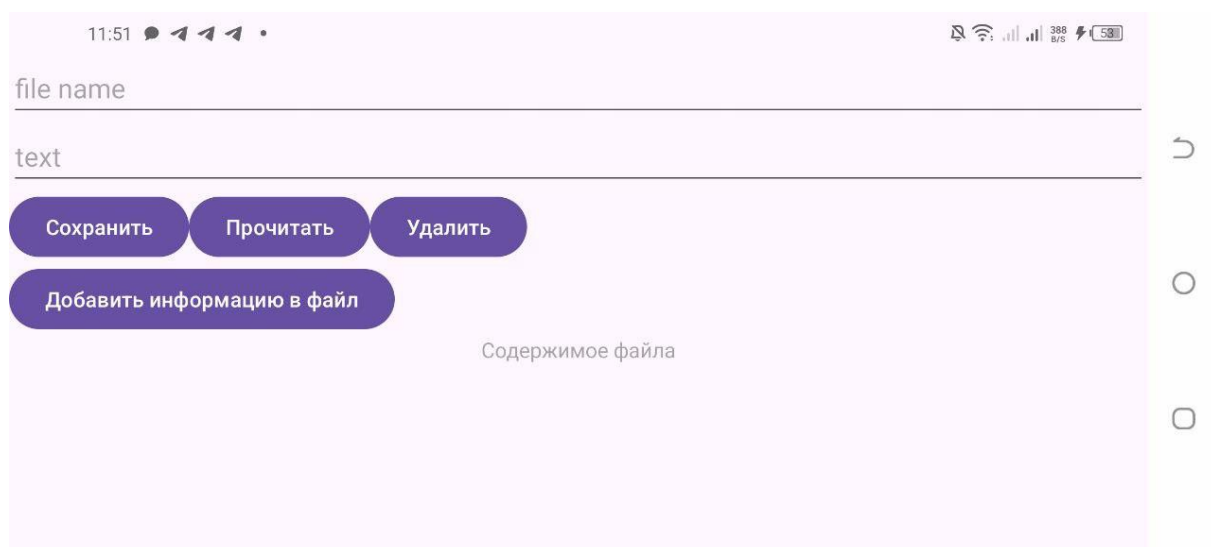


Рисунок 2.2.3 – Горизонтальный вид приложения

2.3 Работа с файлами в различных приложениях

Для начала необходимо задать разметку приложений по записи (рисунок 2.3.1) и чтению (рисунок 2.3.2) файлов.

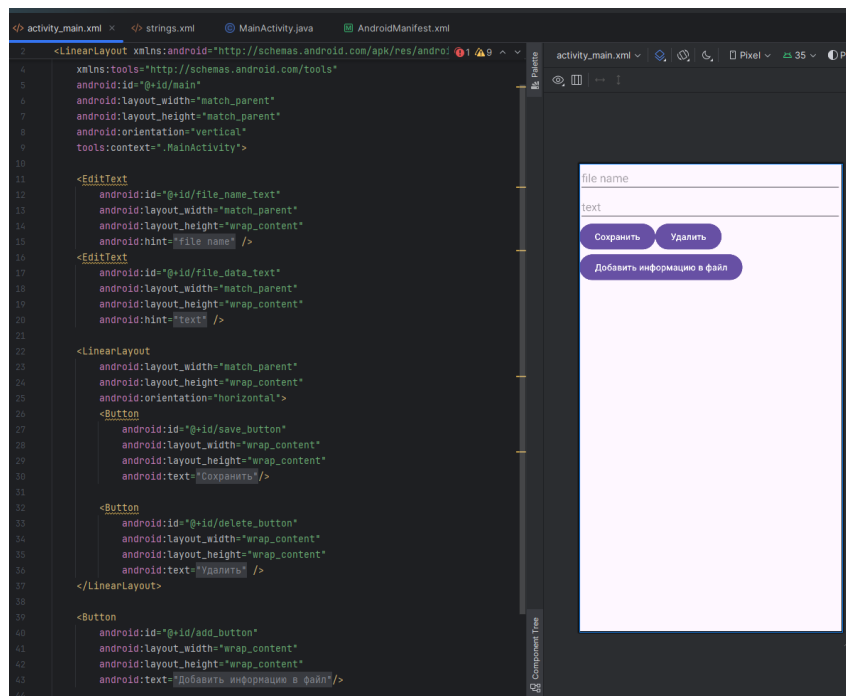


Рисунок 2.3.1 – Графический и кодовый вид приложения для записи файлов

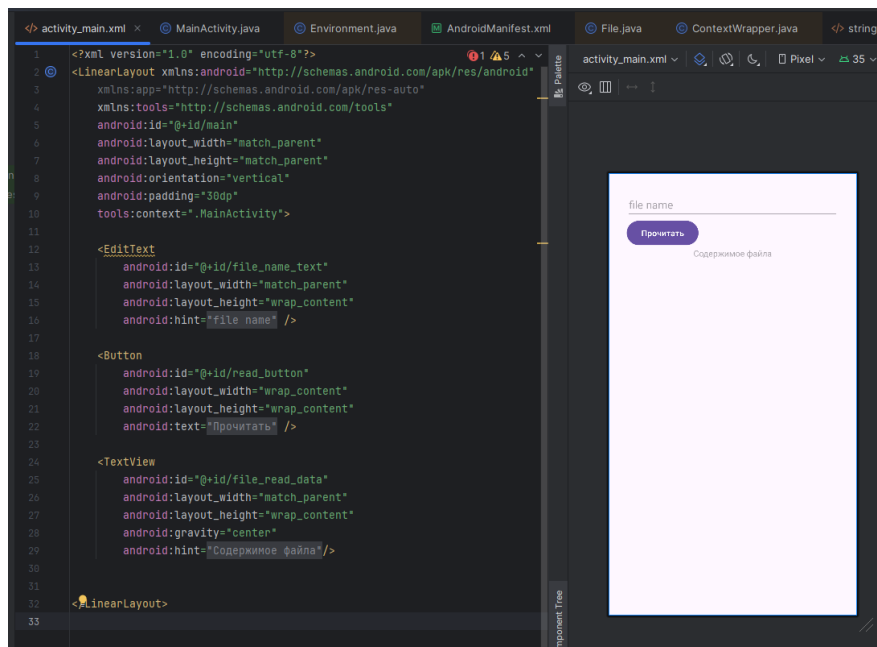
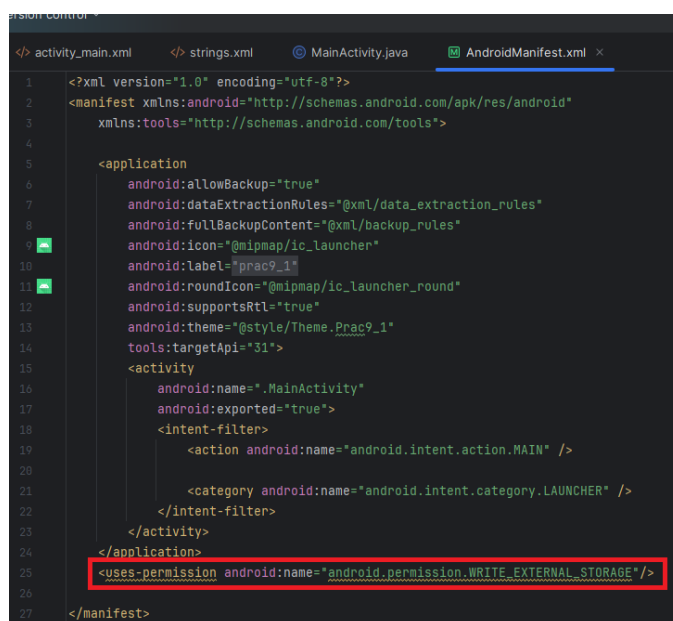


Рисунок 2.3.2 - Графический и кодовый вид приложения для чтения файлов

Для реализации заданного функционала работа будет проводиться с внешним хранилищем приложений. Для записи/чтения файлов из внешних хранилищ других приложений необходимо получить доступ на

непосредственно сами операции в манифестах приложений (рисунки 2.3.3 - 2.3.4).

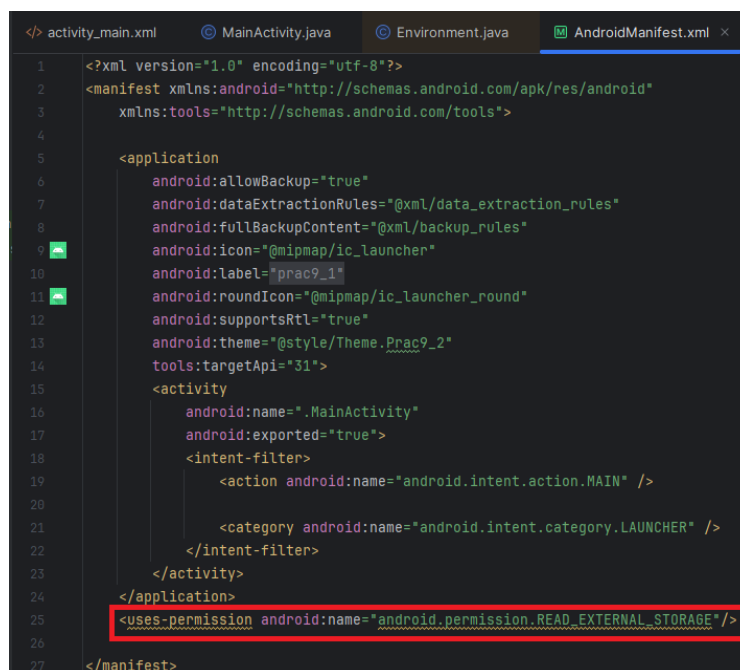


```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/prac9_1"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Prac9_1"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
</manifest>
```

Рисунок 2.3.3 – Получение разрешения на запись файлов во внешнем хранилище



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/prac9_1"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Prac9_2"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
</manifest>
```

Рисунок 2.3.4 – Получение разрешения на чтение файлов из внешнего хранилища

2.3.1 Приложение для записи файлов

Реализация логики данного приложения схожа с приложением обрабатывающим файлы, находящиеся во внутреннем хранилище за тем исключением, что теперь происходит обращение к внешнему общему

хранилищу при помощи метода `getExternalStoragePublicDirectory` (рисунки 2.3.1 – 2.3.3).

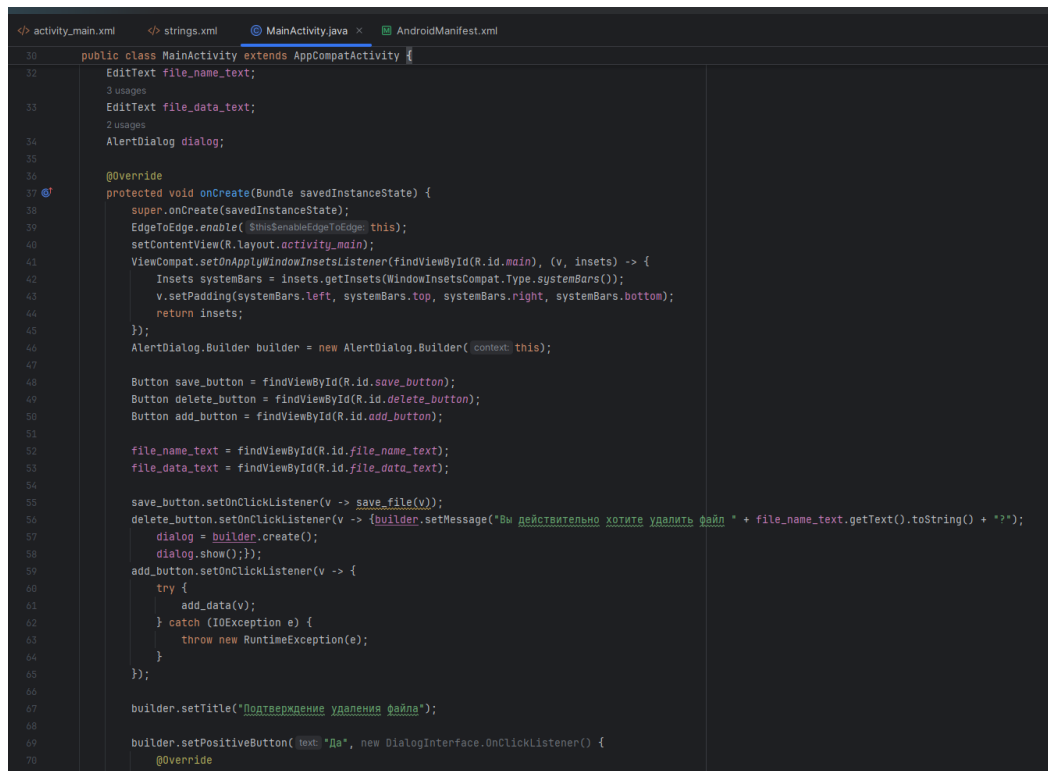


Рисунок 2.3.1 – Часть 1 реализации записи файлов

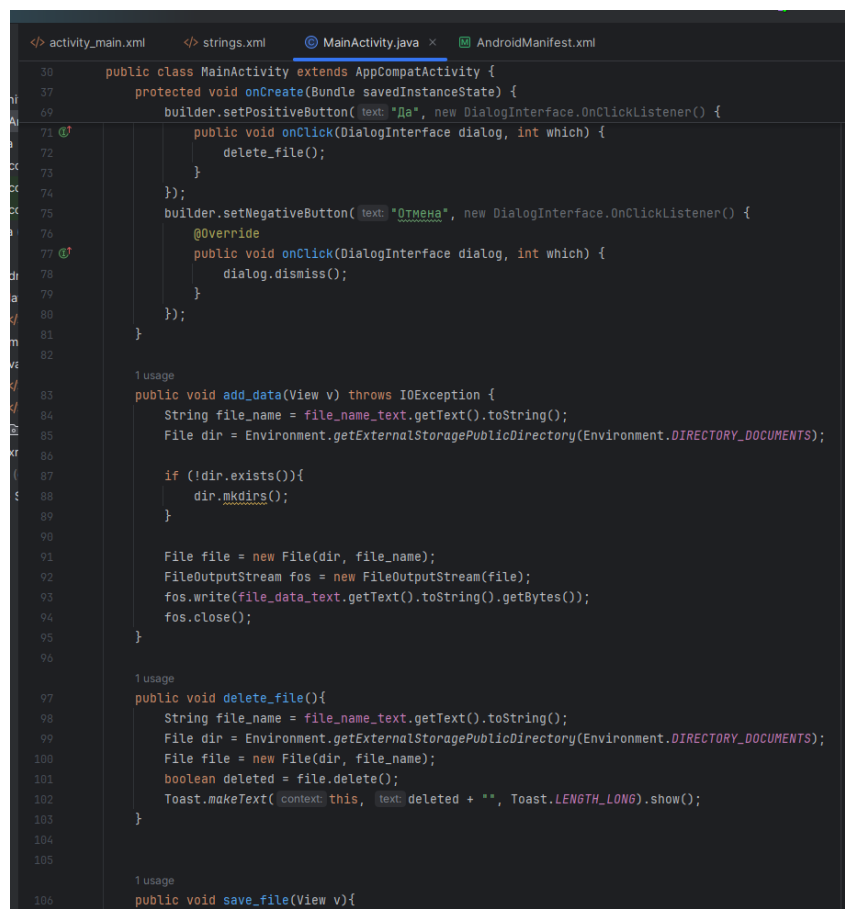


Рисунок 2.3.2 - Часть 2 реализации записи файлов

```

38 public class MainActivity extends AppCompatActivity {
97     public void delete_file(){
102         Toast.makeText(context, this, text: deleted + "", Toast.LENGTH_LONG).show();
103     }
104
105
106     1 usage
107     public void save_file(View v){
108         String file_name = file_name_text.getText().toString();
109         File dir = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);
110
111         if (!dir.exists()){
112             dir.mkdirs();
113         }
114
115         File file = new File(dir, file_name);
116
117         try{
118             if (!file.exists()){
119                 boolean created = file.createNewFile();
120                 if (created){
121                     FileWriter writer = new FileWriter(file);
122                     writer.append(file_data_text.getText().toString());
123                     writer.flush();
124                     writer.close();
125                 }
126             }
127             else{
128                 boolean readable = file.canRead();
129                 boolean writable = file.canWrite();
130             }
131         }catch (IOException e){
132             e.printStackTrace();
133         }
134     }
135 }

```

Рисунок 2.3.3 - Часть 3 реализации записи файлов

Также была произведена проверка корректности работы приложения (рисунки 2.3.4 – 2.3.6).

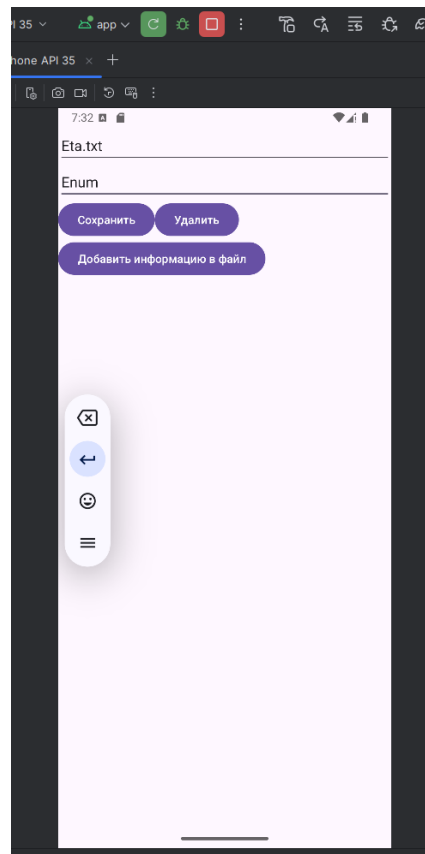


Рисунок 2.3.4 – Создание файла Eta.txt

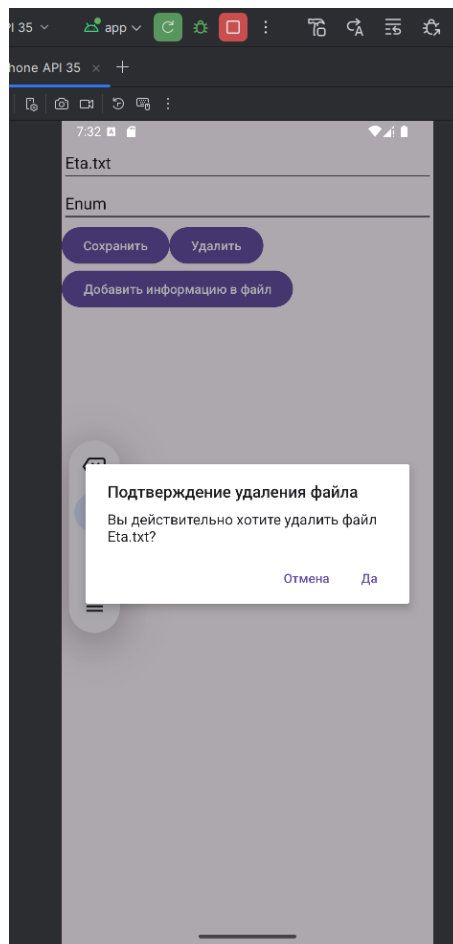


Рисунок 2.3.5 – Удаление файла Eta.txt

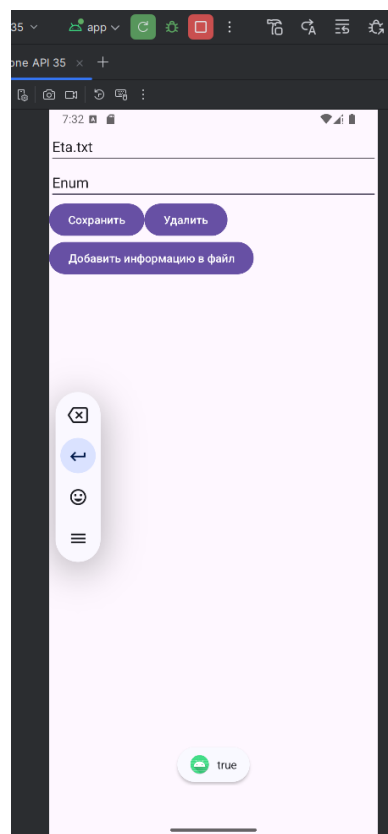
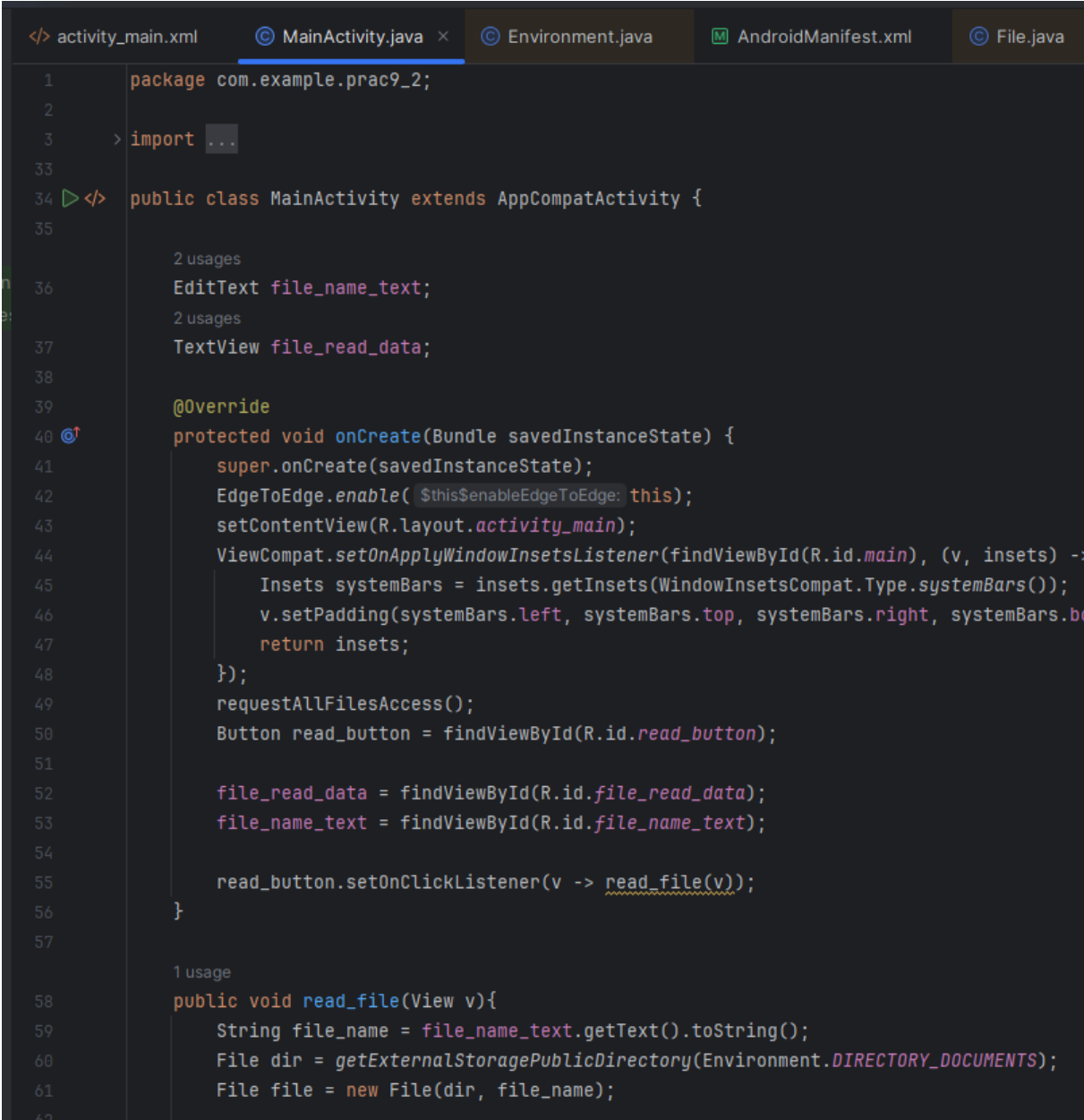


Рисунок 2.3.6 – Итоговый результат удаления файла Eta.txt

Из рисунка 2.3.6 видно, что файл был удалён успешно – это говорит о корректной отработке итогового приложения.

2.3.2 Приложение для считывания данных из файлов

Приложение по считыванию файлов обращается к общему внешнему хранилищу и считывает данные из выбранных файлов, если они существуют и никакие права доступа не были нарушены (рисунок 2.3.2.1 – 2.3.2.2).



```
1 package com.example.prac9_2;
2
3 > import ...
33
34 <> public class MainActivity extends AppCompatActivity {
35
36     2 usages
37     EditText file_name_text;
38     2 usages
39     TextView file_read_data;
40
41     @Override
42     protected void onCreate(Bundle savedInstanceState) {
43         super.onCreate(savedInstanceState);
44         EdgeToEdge.enable(this);
45         setContentView(R.layout.activity_main);
46         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) ->
47             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
48             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
49             return insets;
50         });
51         requestAllFilesAccess();
52         Button read_button = findViewById(R.id.read_button);
53
54         file_read_data = findViewById(R.id.file_read_data);
55         file_name_text = findViewById(R.id.file_name_text);
56
57         read_button.setOnClickListener(v -> read_file(v));
58     }
59
60     1 usage
61     public void read_file(View v){
62         String file_name = file_name_text.getText().toString();
63         File dir = getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);
64         File file = new File(dir, file_name);
```

Рисунок 2.3.2.1 – Часть 1 реализации считывания данных из файлов из общего внешнего хранилища

```

34 public class MainActivity extends AppCompatActivity {
35
36     1 usage
37     public void read_file(View v){
38         String file_name = file_name_text.getText().toString();
39         File dir = getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS);
40         File file = new File(dir, file_name);
41
42         if (file.exists()) {
43             StringBuilder text = new StringBuilder();
44
45             try {
46                 boolean readable = file.canRead();
47                 BufferedReader br = new BufferedReader(new FileReader(file));
48                 String line;
49                 while ((line = br.readLine()) != null) {
50                     text.append(line).append('\n');
51                 }
52                 file_read_data.setText(text);
53             } catch (IOException e) {
54                 e.printStackTrace();
55             }
56         }
57     }
58
59     1 usage
60     private void requestAllFilesAccess() {
61         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
62             try {
63                 Intent intent = new Intent(Settings.ACTION_MANAGE_ALL_FILES_ACCESS_PERMISSION);
64                 startActivity(intent);
65             } catch (Exception e) {
66                 Intent intent = new Intent();
67                 intent.setAction(Settings.ACTION_MANAGE_ALL_FILES_ACCESS_PERMISSION);
68                 startActivity(intent);
69             }
70         }
71     }
72 }

```

Рисунок 2.3.2.2 – Часть 2 реализации считывания данных из файлов из общего внешнего хранилища

Также была проведена проверка итогового функционала приложения (рисунки 2.3.2.3 – 2.3.2.4).

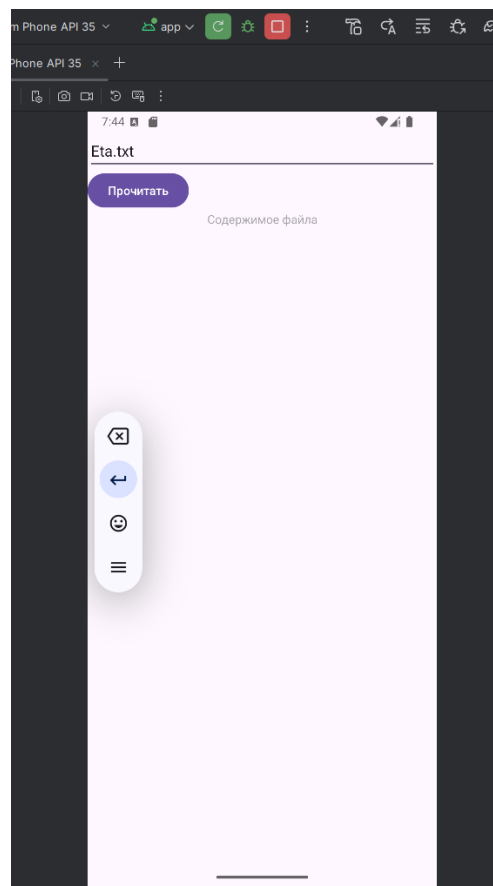


Рисунок 2.3.2.3 – Часть 1 отображения итогового функционала приложения

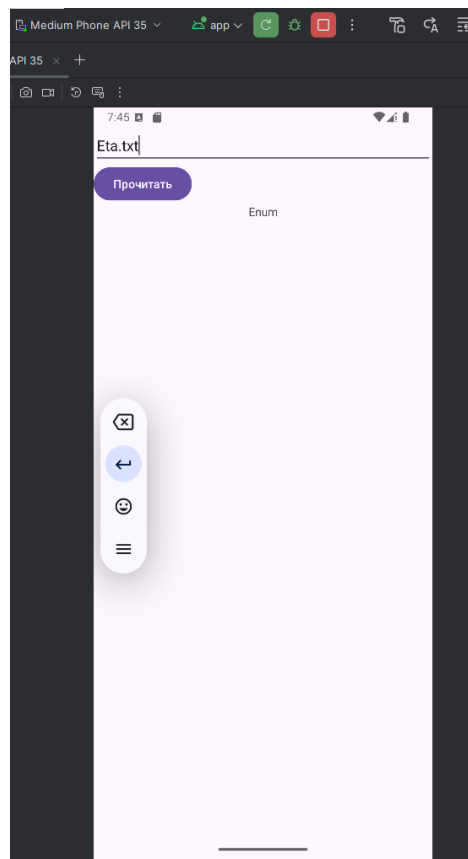


Рисунок 2.3.2.4 – Часть 2 отображения итогового функционала приложения

ЗАКЛЮЧЕНИЕ

В ходе данной практической работы были получены знания по обработке файлов во внешнем и внутреннем хранилищах, а также способ сохранения состояния приложения при смене его ориентации. Полученные знания были закреплены путём выполнения практического задания.